```
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler from keras.models import Sequential
from keras.layers import LSTM, Dense
from sklearn.metrics import mean_squared_error from math import sqrt

# Step 1: Data Collection
# Assuming data is downloaded as a CSV file data = pd.read_csv('historical_stock_data.csv')
data['Date'] = pd.to_datetime(data['Date']) data.set_index('Date', inplace=True)

# Step 2: Data Preprocessing
scaler = MinMaxScaler(feature_range=(0, 1)) data_scaled = scaler.fit_transform(data[['Close']]) #
Create sequences for LSTM
def create_sequences(data, seq_length): sequences, targets = [], []
    for i in range(len(data) - seq_length): sequences.append(data[i:i+seq_length])
targets.append(data[i+seq_length])
    return np.array(sequences), np.array(targets) seq_length = 60
X, y = create_sequences(data_scaled, seq_length) # Split into train and test
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Step 3: Model Training
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(seq_length, 1)), LSTM(50,
return_sequences=False),
    Dense(25),
    Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error') model.fit(X_train, y_train,
batch_size=32, epochs=10) # Step 4: Model Evaluation
predictions = model.predict(X_test)
rmse = sqrt(mean_squared_error(y_test, predictions)) print(f'RMSE: {rmse}')

# Step 5: Visualization
plt.figure(figsize=(10, 6))
plt.plot(data.index[-len(y_test):], scaler.inverse_transform(y_test), label='True Prices')
plt.plot(data.index[-len(predictions):], scaler.inverse_transform(predictions), label='Predicted Prices')
plt.legend()
plt.show()
```

Project 2: Real-Time Face Recognition with OpenCV

Objective:

To develop a system that can identify and recognize faces in real-time using a webcam or video feed. This system can serve various purposes, including